UNIVERSITÀ DI BOLOGNA



Master's Degree in Automation Engineering

Modeling and Simulation of Mechatronic Systems

UR5 ROBOT BOND GRAPH MODEL

Giulio Mazzaroppi, Francesco Cenni, Giacomo Pezzolati

Academic year 25/26

Contents

1 The Link Sub-Model **4**
- 1.1 Ports Description . . . 4
  - 1.1.1 Parameters . . . 4
- 1.2 Ports and parameters . . . 5
- 1.3 Equations of Motion . . . 5
  - 1.3.1 Body Submodel . . . 5
- 1.4 Bond-Graph Structure . . . 5
  - 1.4.1 Gravity Source (MSe) . . . 6
  - 1.4.2 MTF Equations . . . 6
  - 1.4.3 Causality Assignment . . . 6

2 The Joint Sub-Model **8**
- 2.1 Physical Description . . . 8
- 2.2 Ports and Parameters . . . 8
- 2.3 The MTF Element as a DH Transformer . . . 9
- 2.4 Joint Kinematics and Constraint Elements . . . 9
- 2.5 Causality Assignment and Complexity . . . 10

3 The Transformation Sub-Model **12**
- 3.1 Role in the Kinematic Chain . . . 12
- 3.2 Ports & Parameters . . . 12
- 3.3 Equations . . . 12

4 The Harmonic Drive Sub-Model **14**
- 4.1 Introduction . . . 14
- 4.2 Ports & Parameters . . . 14
- 4.3 Bond-Graph Structure . . . 15
  - 4.3.1 Causality Assignment . . . 16

5 The PID Controller Sub-Model **17**
- 5.1 Manipulator Dynamics and Control Objective . . . 17
- 5.2 Control Law . . . 17
- 5.3 Bond-Graph Implementation . . . 18
- 5.4 Gravity Compensation . . . 18
  - 5.4.1 Closed-Form Gravity Torque . . . 18
  - 5.4.2 Implementation in the GravityTerm Block . . . 19

6 Trajectory Generation and Planning **20**
6.1 Overview . . . 20
6.2 Offline Trajectory Generation . . . 21
6.2.1 Waypoint Specification . . . 21
6.2.2 PCHIP Spline Interpolation . . . 22
6.2.3 CSV Export . . . 22
6.3 DataFromFile Block and Reference Erogation . . . 23
6.3.1 DataFromFile Block . . . 23
6.3.2 Pose Assembly . . . 23
6.4 Inverse Kinematics Block . . . 23
6.4.1 UR5 Denavit–Hartenberg Parameters . . . 23
6.4.2 Analytical Solution . . . 24
6.4.3 Singularity Considerations . . . 25
6.5 Phase Unwrapping . . . 25
6.6 Feeding to Low-Level Control . . . 26

7 Overall Robot Architecture **27**
7.1 Hierarchical Structure . . . 27
7.2 Kinematic Chain Assembly . . . 27
7.2.1 Robot Unit . . . 27
7.2.2 Power Bond Chain . . . 28
7.2.3 Signal Chain for Transformations . . . 28
7.3 Global Causality Analysis . . . 29
7.4 Payload Model . . . 29
7.4.1 Overview and Switchable Load . . . 29
7.4.2 Operating Modes . . . 30
7.4.3 Rigid Constraint Between End-Effector and Payload . . . 30
7.4.4 Payload Inertia . . . 31
7.4.5 Conditional Gravity on the Payload . . . 31
7.5 3-D Visualization . . . 31

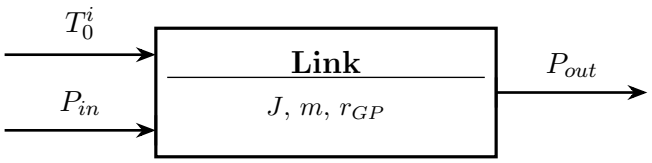
8 Simulation Results **33**
8.1 Test Scenario . . . 33
8.2 3-D Animation . . . 33
8.3 Joint Position Tracking . . . 34
8.4 Joint Torques . . . 35
8.5 Harmonic Drive Elastic Deflection . . . 36
8.6 End-Effector Trajectory: Reference vs. Actual . . . 37
8.7 End-Effector Position: Kinematics vs. Dynamics . . . 39
8.8 Load–End-Effector Constraint Verification . . . 40
8.9 Conclusions . . . 40

Chapter 1

The Link Sub-Model

1.1 Ports Description

Each link of the UR5 is modelled as a rigid body characterised as follows:



Both ports are effort-in flow-out, while the signal port is used to transform the gravity vector from the inertial frame to the body frame of the link.

1.1.1 Parameters

Numerical values for the UR5 are listed in table 1.1.

Table 1.1: UR5 link masses, centres of gravity, and diagonal inertia components.

Link	m [kg]	$r_{GP,x}$ [m]	$r_{GP,y}$ [m]	$r_{GP,z}$ [m]
1	3.700	0.0000	−0.02561	0.00193
2	8.393	0.2125	0.0000	0.11336
3	2.330	0.1500	0.0000	0.02650
4	1.1219	0.0000	−0.00180	0.01634
5	1.1219	0.0000	0.00180	0.01634
6	0.1879	0.0000	0.0000	−0.001159

For what concerns the inertias, matrices for each link are listed below:

$$J_1 = \begin{bmatrix} 0.0067 & 0 & 0 \\ 0 & 0.0064 & 0 \\ 0 & 0 & 0.0067 \end{bmatrix} \text{ kg m}^2, \tag{1.1}$$

$$J_2 = \begin{bmatrix} 0.0149 & 0 & 0 \\ 0 & 0.3564 & 0 \\ 0 & 0 & 0.3553 \end{bmatrix} \text{ kg m}^2, \tag{1.2}$$

$$J_3 = \begin{bmatrix} 0.0025 & 0 & 0.0034 \\ 0 & 0.0551 & 0 \\ 0.0034 & 0 & 0.0546 \end{bmatrix} \text{ kg m}^2, \tag{1.3}$$

$$J_4 = \begin{bmatrix} 0.0012 & 0 & 0 \\ 0 & 0.0012 & 0 \\ 0 & 0 & 0.0009 \end{bmatrix} \text{ kg m}^2, \tag{1.4}$$

$$J_5 = J_4, \qquad J_6 = 0.001 \cdot \mathbf{I}. \tag{1.5}$$

1.2 Ports and parameters

1.3 Equations of Motion

The Newton–Euler equations for link i, written in the body frame attached to the link, are given by. Denoting the generalised momentum as

$$p_{\text{lin}} = m\, v_G, \qquad h_{\text{ang}} = J\, \omega, \tag{1.6}$$

one obtains the state-space form that 20-sim integrates directly:

$$\dot{p}_{\text{lin}} = f^{\text{ext}} + m\, \omega \times v_G, \tag{1.7}$$

$$\dot{h}_{\text{ang}} = \tau^{\text{ext}} + \omega \times J\omega, \tag{1.8}$$

$$v_G = p_{\text{lin}}/m, \tag{1.9}$$

$$\omega = J^{-1} h_{\text{ang}}. \tag{1.10}$$

1.3.1 Body Submodel

The Body submodel is an I-element that integrates the Newton–Euler equations. It is an effort-in flow-out element, meaning that it receives the total wrench acting on the body and outputs the linear and angular velocities at the centre of mass. For these reasons it is connected to a one junction that sums all efforts, after transforming them to the CoM via the MTF elements, and outputs the body velocity that is fed back to the MTFs for the velocity shift. The submodel implements the equations of motion as described above.

1.4 Bond-Graph Structure

The `Link` sub-model exposes two *power ports* of dimension six:

- `wrench_i`: the power applied seen from reference frame of the i-th link. It is seen as the one exiting from joint i and entering link i.

- `wrench_ipp`: it is also seen as the one exiting from link i and entering joint $i+1$. It is the wrench applied by the link on the next joint.

It also receives a signal input `T_next`, the homogeneous transformation, used to orient gravity.

Internally the model contains:

1. Two **MTF** elements (`i_com` and `ipp_com`) that shift the reference from the joint origins to the centre of mass.
2. A **1-junction** that enforces the common-velocity constraint at the centre of mass and sums all efforts.
3. The **Body** I-element that integrates the Newton–Euler equations described above.
4. A **MSe** element that injects the gravitational wrench.

1.4.1 Gravity Source (MSe)

The gravitational force in the inertial frame is $f_g = [0,\, 0,\, -mg]^T$. To express it in the body frame of link i, the rotation matrix $R_i \in SO(3)$ extracted from `T_next` is used:

$$f_g^{\text{body}} = R_i^T \begin{pmatrix} 0 \\ 0 \\ -mg \end{pmatrix}, \qquad \tau_g^{\text{body}} = \mathbf{0}. \tag{1.11}$$

This wrench is injected at the 1-junction so that it acts on the body mass.

1.4.2 MTF Equations

For a displacement vector $r_{Gp} \in \mathbb{R}^3$ from the joint origin to the CoM:

$$v_G = v_{\text{joint}} + \omega \times r_{Gp}, \tag{1.12}$$

$$\tau_{\text{joint}} = \tau_G + r_{Gp} \times f_G. \tag{1.13}$$

These equations are implemented as a modulated transformer in 20-sim, where the modulation matrix is $[r_{Gp}]_\times$.

1.4.3 Causality Assignment

Both power ports carry *fixed integral causality*. This means:

- Efforts (forces/torques at the joints) are *given* by the connected joints.
- Velocities (flows) are *computed* by the body integrator.

The 1-junction has one element with *effort-out* causality (the Body I-element) and all others with *effort-in* causality (the two MTF connections and the MSe source).

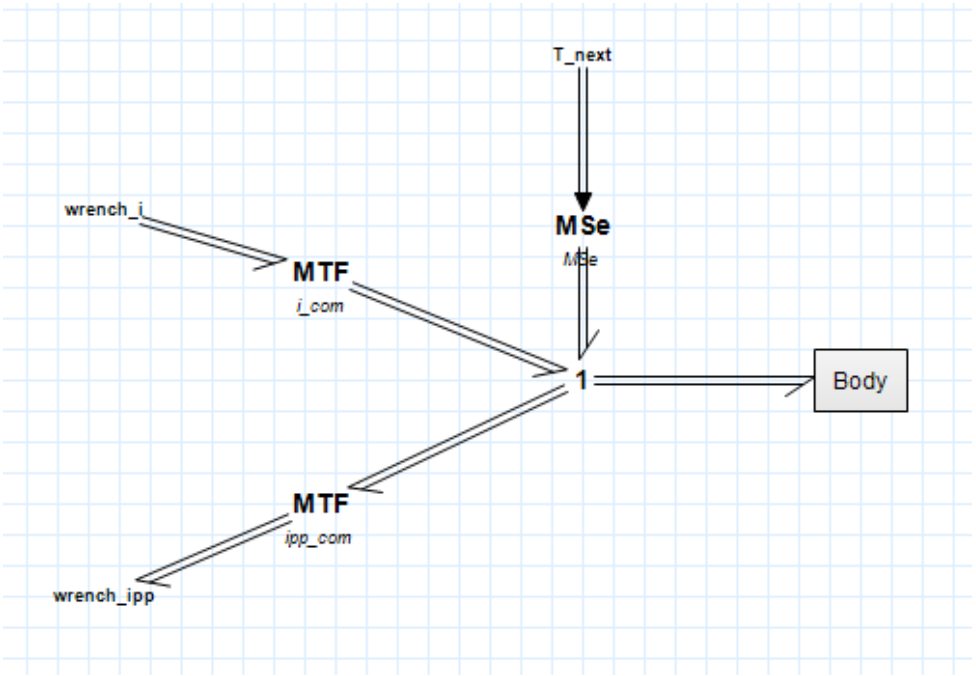


Figure 1.1: Bond-graph diagram of the `Link` sub-model.

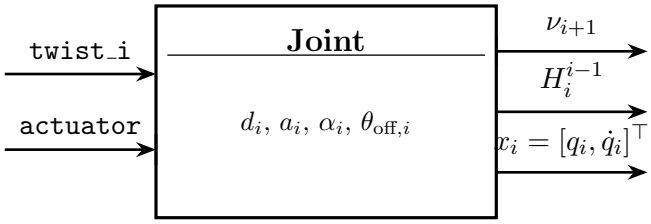
Chapter 2

The Joint Sub-Model

2.1 Physical Description

Each of the six joints of the UR5 is a revolute joint. In the bond-graph model, the `Joint` sub-model is responsible for:

1. Performing the DH frame transformation between the inbound and outbound twist ports.
2. Extracting the scalar joint position and velocity $(q_i, \dot{q}_i) \quad \forall i = 1, \ldots, 6$
3. Receiving the scalar actuator torque $\tau_i \quad \forall i = 1, \ldots, 6$ and transmitting it to the mechanical chain.



2.2 Ports and Parameters

Table 2.1: `Joint` sub-model ports.

Port	Type	Description
ν_i	power in (fixed-out causality)	Twist of frame $i-1$
ν_{i+1}	power out (fixed-out causality)	Twist of frame i
τ_i	power in (fixed-in causality)	Actuator scalar torque bond
H_i^{i-1}	signal out $[4 \times 4]$	DH homogeneous transformation H_i^{i-1}
x_i	signal out $[2 \times 1]$	Joint state $[q_i,\ \dot{q}_i]^T$

DH parameters stored in the `Joint` sub-model are d_i, a_i, α_i, and the offset $\tilde{q}_i$. These are set for each of the six joints according to 2.2.

Table 2.2: UR5 Denavit–Hartenberg parameters.

Joint	d_i [m]	a_i [m]	α_i [rad]	$\tilde{q}_i$ [rad]
Base (1)	0.089159	0.0	$\pi/2$	0
Shoulder(2)	0.0	-0.425	0	0
Elbow (3)	0.0	-0.39225	0	0
Wrist1 (4)	0.10915	0.0	$\pi/2$	0
Wrist2 (5)	0.09465	0.0	$-\pi/2$	0
Wrist3 (6)	0.0823	0.0	0	0

2.3 The MTF Element as a DH Transformer

The core of the `Joint` model is an MTF element that implements the DH transformation. Given the joint angle $\theta_i = q_i + \tilde{q}_i$ and the stored parameters, the rotation matrix R_i and translation P_i are computed as follows:

$$R_i = \begin{pmatrix} \cos\theta_i & -\sin\theta_i \cos\alpha_i & \sin\theta_i \sin\alpha_i \\ \sin\theta_i & \cos\theta_i \cos\alpha_i & -\cos\theta_i \sin\alpha_i \\ 0 & \sin\alpha_i & \cos\alpha_i \end{pmatrix}, \tag{2.1}$$

$$P_i = \begin{pmatrix} a_i \cos\theta_i \\ a_i \sin\theta_i \\ d_i \end{pmatrix}. \tag{2.2}$$

The MTF enforces power continuity by transforming twists and wrenches:

$$\tau_{\text{ipp}} = R_i^T \tau_i, \tag{2.3}$$

$$f_{\text{ipp}} = R_i^T f_i - R_i^T (P_i \times \tau_i), \tag{2.4}$$

$$\omega_i = R_i\, \omega_{\text{ipp}}, \tag{2.5}$$

$$v_i = P_i \times R_i\, \omega_{\text{ipp}} + R_i\, v_{\text{ipp}}. \tag{2.6}$$

Simultaneously, the MTF outputs the homogeneous transformation matrix T_i as a signal to be used by the 3D animation tool for the visual representation of the kinematic chain.

2.4 Joint Kinematics and Constraint Elements

A revolute pair restricts five of the six possible degrees of freedom between two adjacent links, allowing only a relative rotation about a single axis. In the adopted bond-graph topology, this physical restriction is modeled by splitting the 6-dimensional twist variables using `PowerDemux` elements and selectively applying velocity constraints. This allows to:

- **Split the 3D translational velocity components** into v_{i_x}, v_{i_y}, and v_{i_z}.
- **Split the 3D angular velocity components** into ω_{i_x}, ω_{i_y}, and ω_{i_z}.

Assuming the z-axis as the rotation axis, the constraints are enforced as follows:

- **Translational Constraints**: The relative linear velocities along the x, y, and z axes must be zero. This is achieved via a `0-junction` connected to a `Vel_constr` element. The constraint acts as an infinite impedance, effectively outputting the exact reaction forces required to maintain zero translational flow between the links.
- **Rotational Constraints (x and y axes)**: Similarly, rotations about the x and y axes are blocked. Another `0-junction` connected to the `Omega_constr` element imposes a zero-flow condition, generating the necessary reaction bending moments.
- **Actuated Rotational Degree of Freedom (z axis)**: The z-component is the only unconstrained axis. A dedicated `0-junction` acts as the summation point for the efforts (torques) acting on this DoF. The `actuator` port, which receives the control torque computed by the controller, connects directly to this junction.

Finally, the joint state variables are extracted from the power flow of this unconstrained axis: a **FlowSensor** reads the scalar rotational velocity $\dot{q}_i$ (which corresponds to ω_z), and an **Integrator** computes the joint position $q_i = \int \dot{q}_i \, dt$ to be fed into the MTF for the DH transformation.

2.5 Causality Assignment and Complexity

The joint sub-model presents the most complex causality situation in the entire robot model:

- The inbound port `twist_i` has *fixed effort-out* causality: the joint *imposes* force/torque on the previous link and *receives* velocity from it.
- The outbound port `twist_ipp` has *fixed effort-out* causality as well: the joint imposes force/torque on the next link.
- The MTF has a *causality constraint*: the two sides must have opposite causality (one effort-in, one effort-out).
- The constraint elements (`Vel_constr` for translational DoFs, `Omega_constr` for xy-rotation) set effort from flow via an algebraic constraint equation, implying *derivative causality* on the constrained directions.
- The `actuator` port has *fixed-in causality*: the motor effort is imposed externally, so the 0-junction for q_z has *one effort-in* element (the actuator).

The combination of fixed causality on both external power ports and the internal MTF constraint means that *no free integral causality* can be assigned to the internal inertia elements of the joint body; instead, these joints rely on the inertia of the connected links. This is the correct physical interpretation: the joint itself has zero inertia in this model.

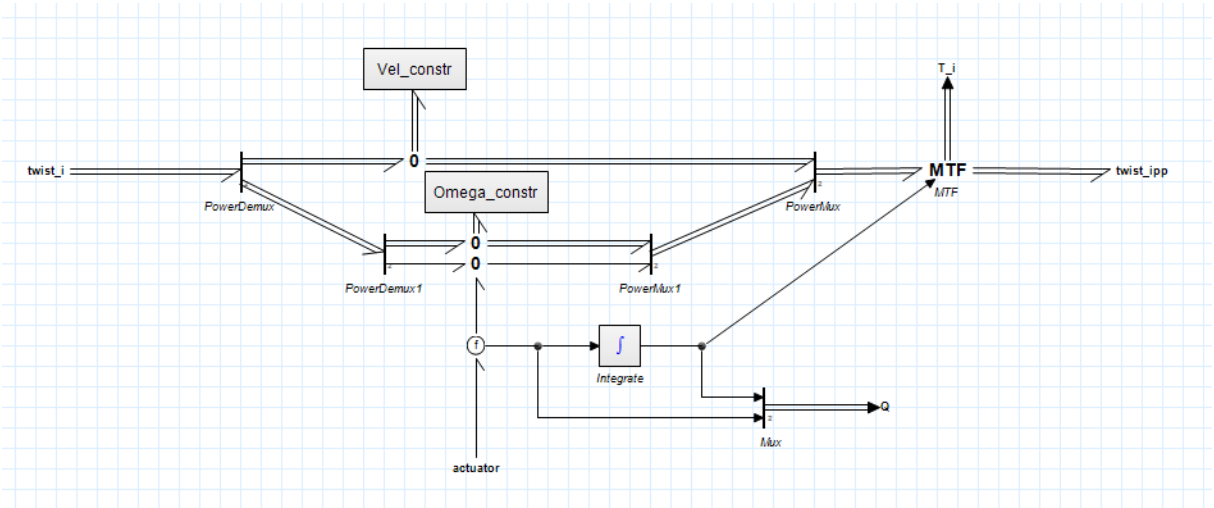


Figure 2.1: Bond-graph diagram of the `Joint` sub-model.

Chapter 3

The Transformation Sub-Model

3.1 Role in the Kinematic Chain

The `Transformation` sub-model is a *signal-only* block that computes the cumulative homogeneous transformation from the previous frame $i-1$ to the current frame i.

The transformation chain, as depicted in Figure 3.1, computes the total transformation with respect to the base frame, while also providing useful intermediate transformations.

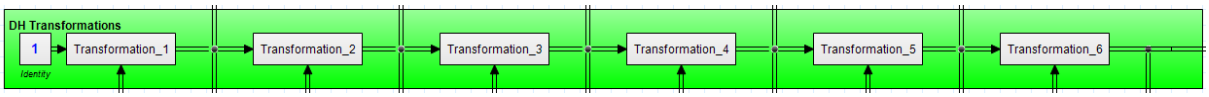
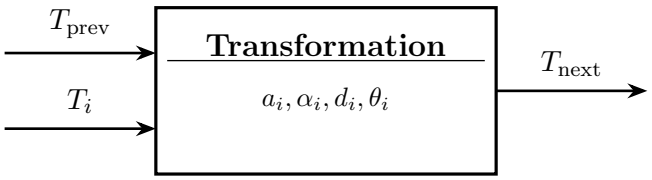


Figure 3.1: Chain of transformations

3.2 Ports & Parameters

The `Transformation` sub-model's port structure is as follows:



3.3 Equations

Given the absolute transformation T_{prev} up to frame $i-1$ and the incremental DH transformation T_i provided by the `Joint` block, the absolute transformation is updated as:

$$T_{\text{next}} = T_{\text{prev}} \cdot T_i. \tag{3.1}$$

The transformation from a general frame i to the base frame is given by the product of the sequential transformations:

$${}^0T_i = {}^0T_1 \cdot {}^1T_2 \ldots {}^{i-1}T_i$$

Furthermore, within each transformation block, the following variables are computed:

- **Position of the DH frame with respect to the base frame:** ${}^0p_i = T[1:3,4]$, which is utilized to define the position of the DH frames in the 3D visualization.
- **Orientation of the DH frame with respect to the base frame:** ${}^0R_i = T[1:3,1:3]$. This rotation matrix dictates the orientation of the DH frame in the 3D visualization and is also transposed to re-orient the gravity vector from the base frame to the DH frame within the **MSe** element of the `Link` block.
- **Position of a relative point on link i with respect to the base frame:** ${}^0p_{\text{body},i} = {}^0R_i \, {}^ip_{\text{body},i} + {}^0p_i$, used to offset the link body inside the 3D visualization.
- **Orientation of link i with respect to the base frame:** ${}^0R_{\text{body},i} = {}^0R_i \, {}^iR_{\text{body},i}$, used to properly orient the link body in the 3D visualization.

Lastly, the complete transformation matrix T_{next} is utilized inside the gravity compensation block to compute the directional axes z_i, the positions of the DH frames p_i, and the centers of mass $p_{\text{com},i}$, all evaluated with respect to the base frame.

Chapter 4

The Harmonic Drive Sub-Model

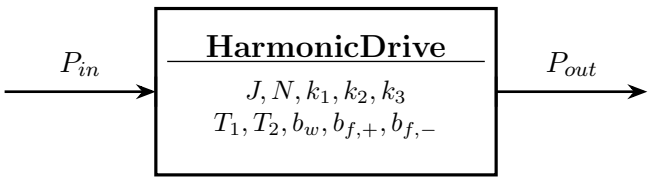
4.1 Introduction

Robotic joints are not directly driven: a gearbox with a high reduction ratio sits between the motor shaft and the output link. In the UR5, *harmonic-drive* (strain-wave) gearboxes are used. These components exhibit several non-ideal characteristics that introduce dynamic effects that affect the closed-loop performance:

1. High gear ratio with near-zero backlash.
2. Non-linear torsional stiffness (the flex-spline deforms elastically).
3. Viscous friction at the input shaft.
4. Asymmetric Coulomb friction (static friction differs for positive and negative rotation).
5. Motor rotor inertia reflected through the gear ratio.
6. Internal structural damping within the flex-spline.

4.2 Ports & Parameters

The `HarmonicDrive` sub-model's port structure is as follows:



The input port `motor` receives the motor torque and angular velocity (effort-in, flow-out), while the output port `harmonic_drive` provides the joint torque and output shaft velocity (effort-out, flow-in).

4.3 Bond-Graph Structure

The `HarmonicDrive` sub-model has:

- `motor`: scalar power port input.
- `harmonic_drive`: scalar power port output.

The topology is shown schematically in the bond graph below. The key elements are:

Gearbox inertia (I element). The harmonic drive inertia J_{rotor} stores kinetic energy:

$$J\,\dot{\omega}_m = \tau_m - \tau_{\text{friction}} - \tau_{\text{elastic}}/N, \tag{4.1}$$

where ω_m is the motor angular velocity and N is the gear ratio (negative due to sign convention, default $N = -0.01$, i.e. ratio of 100).

Gear ratio (TF element). The transformer element with ratio N enforces:

$$\tau_m = N\,\tau_{\text{out}}, \qquad \omega_{\text{out}} = N\,\omega_m. \tag{4.2}$$

Torsional stiffness (C element – smooth cubic). The flex-spline introduces a nonlinear torsional compliance. The constitutive law is implemented as a smooth cubic polynomial that interpolates the three stiffness breakpoints:

$$\tau_s(\varphi) = a\,\varphi^3 + b\,\varphi, \tag{4.3}$$

where $\varphi = \int(\omega_{\text{out}} - \omega_{\text{load}})\,dt$ is the elastic twist of the flex-spline, and the coefficients a and b are computed from the three-zone stiffness data via:

$$\varphi_1 = \frac{T_1}{k_1}, \qquad \varphi_2 = \varphi_1 + \frac{T_2 - T_1}{k_2}, \tag{4.4}$$

$$\Delta = \varphi_1\,\varphi_2\big(\varphi_1^2 - \varphi_2^2\big), \tag{4.5}$$

$$a = \frac{\varphi_2\,T_1 - \varphi_1\,T_2}{\Delta}, \qquad b = \frac{-\varphi_2^3\,T_1 + \varphi_1^3\,T_2}{\Delta}, \tag{4.6}$$

Viscous and Coulomb friction at the motor shaft (R element). At the motor-side 1-junction (input shaft), a combined resistance element captures both viscous damping and asymmetric Coulomb friction:

$$\tau_f(\omega) = \frac{b_{f,+} - b_{f,-}}{2} \cdot \frac{2}{\pi}\arctan\!\Big(\omega - \tan\!\Big(\frac{\pi}{2}\cdot\frac{b_{f,+} + b_{f,-}}{b_{f,+} - b_{f,-}}\Big)\Big) + \frac{b_{f,+} + b_{f,-}}{2} + b_w\,\omega, \tag{4.7}$$

with $b_{f,+} = 0.008$ Nm and $b_{f,-} = -0.007$ Nm, and $b_w = 6.4 \times 10^{-4}$ Nm s/rad. The arctan regularisation avoids a discontinuity at zero velocity and ensures numerical stability.

Asymmetric Coulomb friction at the output shaft (R element). An identical resistance formulation is applied at the output-side 1-junction with the same parameters ($b_{f,+}$, $b_{f,-}$, b_w), capturing friction at the output bearing of the strain-wave gear.

Structural damping of the flex-spline (R element). A third resistance element is placed at the compliance 1-junction, in parallel with the torsional spring C element. This models the internal structural damping of the flex-spline itself: as the flex-spline elastically deforms, part of the stored energy is dissipated through material hysteresis. The resistance is governed by the same smooth regularised formula:

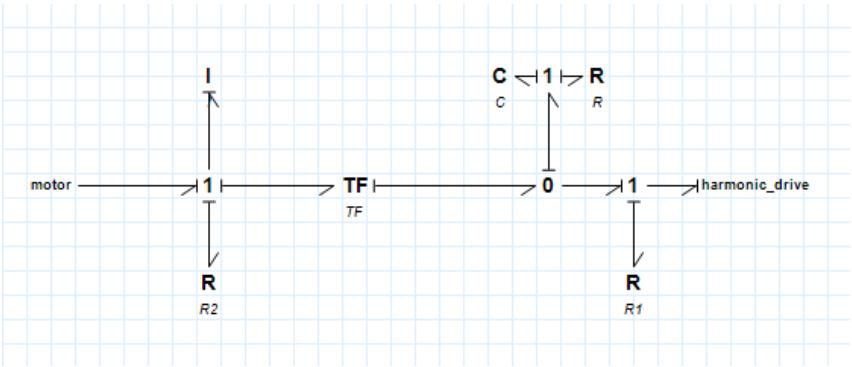
$$\tau_d(\dot{\varphi}) = B_s \cdot \frac{2}{\pi} \arctan(\dot{\varphi}) + b_w \, \dot{\varphi}, \tag{4.8}$$

where $\dot{\varphi} = \omega_{\text{out}} - \omega_{\text{load}}$ is the relative angular velocity across the flex-spline, $B_s = 0.4$ Nm is the symmetric Coulomb-like structural damping coefficient (i.e. $B_{s,+} = 0.4$ Nm, $B_{s,-} = -0.4$ Nm), and b_w is the same viscous coefficient shared with the shaft friction elements. Because this element is on the compliance branch, its effort opposes the rate of elastic deformation, adding dissipation proportional to how rapidly the flex-spline is being twisted. This element is essential to damp the high-frequency oscillations associated with the elastic flex-spline mode that would otherwise persist indefinitely under a purely conservative C element.

4.3.1 Causality Assignment

The `motor` port is fixed *effort-in*: the motor torque source imposes effort. The `harmonic_drive` port is fixed *effort-out*: the transmission output imposes torque on the joint. Within the sub-model:

- The 1-junction at the motor shaft has the I-element as the *effort-out* element (integral causality), and the TF and R elements as effort-in elements.
- The 0-junction at the output shaft is the sum point for the elastic torque from the compliance branch, the structural damping torque, and the output Coulomb friction.
- The C element is in integral causality: $\dot{\varphi} = f_C$, $e_C = \tau_s(\varphi)$.
- The structural damping R on the compliance branch shares the same 1-junction as the C element; it therefore sees the same flow $\dot{\varphi}$ and contributes an additional dissipative effort in parallel with the spring torque.



Chapter 5

The PID Controller Sub-Model

5.1 Manipulator Dynamics and Control Objective

The equations of motion of an n-link robot manipulator can be written in the joint space as a set of nonlinear, coupled differential equations:

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + D\dot{q} + g(q) = \tau \tag{5.1}$$

where $n = 6$ for the UR5 manipulator. The terms appearing in eq. (5.1) represent:

- $q \in \mathbb{R}^{6\times1}$ is the vector of joint positions (angular displacements) [rad].
- $\dot{q} \in \mathbb{R}^{6\times1}$ is the vector of joint velocities [rad/s].
- $\ddot{q} \in \mathbb{R}^{6\times1}$ is the vector of joint accelerations [rad/s^2].
- $M(q) = M(q)^\top \succeq 0 \in \mathbb{R}^{6\times6}$ is the joint-space inertia matrix [kg $\cdot$ m^2].
- $C(q,\dot{q}) \in \mathbb{R}^{6\times6}$ is the matrix accounting for Coriolis and centrifugal forces [kg$\cdot$m^2/s].
- $D = D^\top \succeq 0 \in \mathbb{R}^{6\times6}$ is the matrix of viscous friction coefficients [kg $\cdot$ m^2/s].
- $g(q) \in \mathbb{R}^{6\times1}$ is the vector of generalized torques due to gravity [N $\cdot$ m].
- $\tau \in \mathbb{R}^{6\times1}$ is the vector of generalized torques applied by the joint actuators [N $\cdot$ m].

The control objective is to follow a kinematic trajectory given with an unknown mass load.

5.2 Control Law

To fulfill the control objective, the centralized control torque vector τ is chosen as:

$$\tau = K_p(q^* - q) - K_d\dot{q} + K_i(q^* - q) + g(q) \tag{5.2}$$

where:

- $K_p = K_p^\top \succeq 0 \in \mathbb{R}^{6\times6}$ is the matrix containing the proportional gains [N $\cdot$ m/rad].
- $K_d = K_d^\top \succeq 0 \in \mathbb{R}^{6\times6}$ is the matrix containing the derivative gains [N $\cdot$ m $\cdot$ s/rad].

- $K_i = K_i^\top \succeq 0 \in \mathbb{R}^{6\times 6}$ is the matrix containing the integral gains $[\mathrm{N}\cdot\mathrm{m}\cdot\mathrm{s/rad}]$.

The control law consists of a proportional action tracking the regulation error, a derivative action acting directly on the joint velocity to inject damping since we are not using inverse dynamics, an integral action to compensate for the unknown mass load and a nonlinear feedforward term providing compensation for the gravitational torques of the robot.

5.3 Bond-Graph Implementation

Each `PID` block within the 20-sim library acts as a single-joint decentralized execution unit. It processes continuous signal flows and converts them into an output command. The block maps the control law component-wise for a single joint index i:

$$\tau_i = K_{p,i}(q_i^* - q_i) - K_{d,i}\dot{q}_i + K_{i,i}(q_i^* - q_i) + \tau_{g,i} \tag{5.3}$$

The sub-model handles the following external interfaces:

- It receives the reference joint angle q_i^* from the inverse kinematics block.
- It receives the vector signal $\mathtt{Q} = [q_i, \dot{q}_i]^T$ from the physical `Joint` block.
- It receives the scalar gravity feedforward signal $\mathtt{gravity} = \tau_{g,i}$ from the centralized computing unit.
- It outputs the scalar signal `tau` directed to the effort source modulating the joint's actuator.

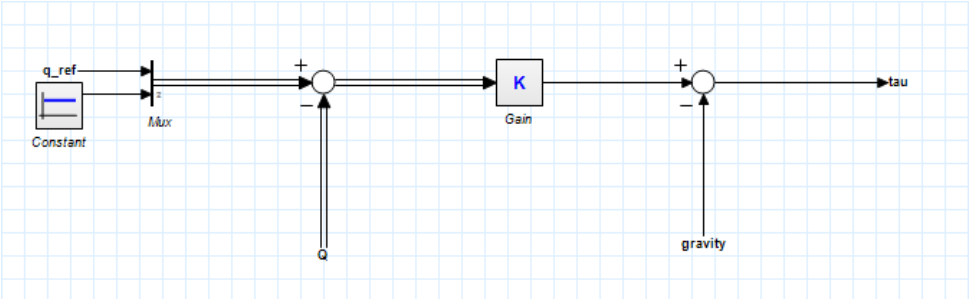


Figure 5.1: Internal structure of the `PD` sub-model in 20-sim.

5.4 Gravity Compensation

5.4.1 Closed-Form Gravity Torque

The gravitational vector $g(q)$ is generated from the potential energy distribution of the multi-body system. For any joint i, the required static torque $\tau_{g,i}$ is determined by accumulating the moments produced by all subsequent links $j \geq i$.

The mass of the load is considered totally unknown as if it was not present. The generalized gravity torque is evaluated through the vector cross-product equation:

$$\tau_{g,i} = \sum_{j=i}^{6} m_j \, g^T \left(z_{i-1} \times (p_{G,j}^0 - p_{i-1}^0) \right), \tag{5.4}$$

where:

- $g = [0,\, 0,\, -9.81]^T \in \mathbb{R}^{3\times 1}$ is the static acceleration gravity vector defined in the base reference frame.
- $z_{i-1} \in \mathbb{R}^{3\times 1}$ is the orientation vector of the joint actuation axis, extracted as the third column of the homogeneous transformation matrix T_{i-1}.
- $p^0_{G,j} \in \mathbb{R}^{3\times 1}$ is the coordinates vector specifying the absolute position of the center of mass of link j.
- $p^0_{i-1} \in \mathbb{R}^{3\times 1}$ is the absolute position vector of the origin of coordinate frame $i-1$.

5.4.2 Implementation in the GravityTerm Block

The `GravityTerm` block coordinates the computation of the feedforward variables across all joints. It operates entirely as a mathematical *signal* block, meaning it contains no power bonds and does not impose causality constraints during the model resolution.

The structural transformations T_1 through T_6 enter the block as 4×4 matrices from the kinematic transformation chain. For each link index j, the translation parameters and spatial vectors are mapped according to the following operations:

$$p^0_j = T_j[1{:}3,\, 4], \tag{5.5}$$

$$p^0_{G,j} = T_j[1{:}3,\, 1{:}3] \cdot r_{\mathrm{gp},j} + p^0_j, \tag{5.6}$$

$$z_j = T_j[1{:}3,\, 3]. \tag{5.7}$$

The block computes the individual scalar outputs `tau_1` through `tau_6` executing the vector additions defined in eq. (5.4). The outputs are packed into the vector signal `tau_g` $\in \mathbb{R}^{6\times 1}$ and forwarded to the respective controllers.

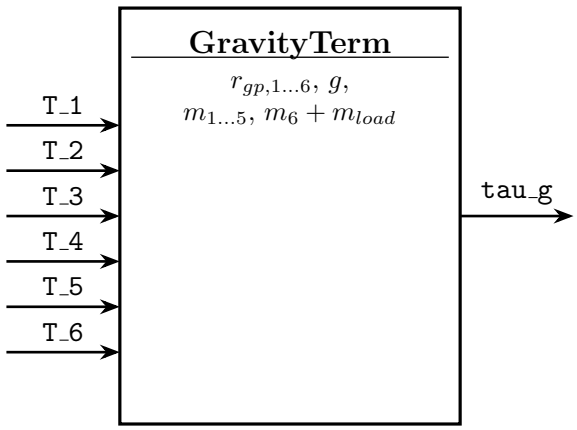


Figure 5.2: Signal interface of the `GravityTerm` sub-model in 20-sim.

Chapter 6

Trajectory Generation and Planning

6.1 Overview

The complete trajectory-tracking stack of the UR5 model can be decomposed into three levels:

1. **Offline trajectory generation**: We implemented a Python script that generates the following files and exports them to CSV
   - A smooth Workspace reference trajectory (6D) for the end-effector, defined by a sequence of waypoints and interpolated using `PCHIP` cubic splines.
   - A load condition signal that indicates whether the end-effector is carrying a load or not, based on the inter-waypoint segments.
2. **Online reference erogation and Workspace-to-Joint mapping**: the CSV is read at each simulation step and the desired end-effector pose is converted to a homogeneous transformation matrix and then mapped to joint angles by the closed-form IK solver.
3. **Joint reference conditioning**: the raw IK output is phase-unwrapped to remove 2π discontinuities, and the resulting continuous joint references are fed to the six PD+G controllers.

The UR5 joints angle is provided by an Integrator block, which does not enforce any bounds on the joint angles. This means that the joint angles can grow indefinitely in either direction, and the controller must be designed to handle this unboundedness. The phase unwrapping step is crucial to ensure that the joint references remain continuous and do not exhibit jumps of 2π when crossing the $\pm\pi$ boundary.

Figure 6.1 illustrates the signal flow from the offline planner through to the low-level controllers.

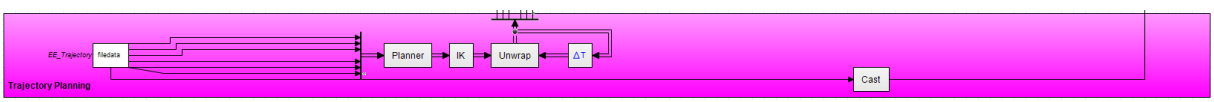


Figure 6.1: Trajectory planning and control architecture.

6.2 Offline Trajectory Generation

6.2.1 Waypoint Specification

The end-effector trajectory is defined by a set of $N_w = 11$ waypoints in the six-dimensional Cartesian space $[X,\ Y,\ Z,\ \phi,\ \theta,\ \psi]^\top$, where (X, Y, Z) is the position of the tool-center point in the world frame and (ϕ, θ, ψ) are the XYZ Cardan angles (yaw, pitch, roll) of the end-effector orientation.

The waypoints are sampled at uniformly spaced times over a time horizon of $T_{\text{end}} = 100$ seconds, so that the k-th waypoint is defined at time t_k as:

$$t_k = k \cdot \frac{T_{\text{end}}}{N_w - 1}, \qquad k = 0, 1, \ldots, N_w - 1. \tag{6.1}$$

The waypoints are listed in table 6.1 and encode a motion sequence in which the robot first moves from its rest posture to a sequence of pick-and-place positions.

- **Pre-grasp**: The first three waypoints ($k = 0, 1, 2$) define a motion from the initial pose to a pre-grasp position above the first load.
- **Pick**: The $k = 2$ waypoint encodes the load position and the grasping action (load attached at t_3).
- **Post-grasp**: The $k = 3$ waypoint define the post-grasp position, where the load is lifted up.
- **Transport**: The waypoints from $k = 4$ to $k = 5$ define a transport motion to the pre-release pose.
- **Pre-release**: The $k = 6$ waypoint encodes the pre-release position, where the load is held above the release point.
- **Release**: The $k = 7$ waypoint encodes the release position, where the load is detached at t_7.
- **Post-release**: The waypoint $k = 8$ defines the post-release position, where the end-effector is lifted up after releasing the load.
- **Return to rest**: The last two waypoints ($k = 9, 10$) define a motion back to the initial rest posture.

Table 6.1: Cartesian waypoints for the end-effector trajectory.

k	X [m]	Y [m]	Z [m]	ϕ [rad]	θ [rad]	ψ [rad]	cond[bool]
0	-0.72	-0.11	0.007	$\pi/2$	0	0	`False`
1	0.00	0.50	0.200	π	0	0	`False`
2	0.30	0.00	0.200	π	0	0	`False`
3	0.30	0.00	0.050	π	0	0	`True`
4	0.30	0.00	0.200	π	0	0	`True`
5	0.00	-0.50	0.200	π	0	0	`True`
6	-0.30	0.00	0.200	π	0	0	`True`
7	-0.30	0.00	0.050	π	0	0	`True`
8	-0.30	0.00	0.200	π	0	0	`False`
9	0.00	0.30	0.500	π	0	0	`False`
10	0.00	0.30	0.500	π	0	0	`False`

A boolean condition array `cond_sp` $\in \{0,1\}^{N_w}$ assigns a load status to each inter-waypoint segment: segments $[t_3, t_7]$ have `Cond` $= 1$ (load attached), all others have `Cond` $= 0$ (free end-effector). The condition is held constant within each segment.

6.2.2 PCHIP Spline Interpolation

The six Cartesian channels are independently interpolated using a *Piecewise Cubic Hermite Interpolating Polynomial* (PCHIP):

$$\mathbf{x}(t) = \text{PCHIP}(\{t_k\},\, \{\mathbf{x}_k\},\, t)\,, \qquad \mathbf{x} = [X,\, Y,\, Z,\, \phi,\, \theta,\, \psi]^T. \tag{6.2}$$

PCHIP is preferred over a natural cubic spline because it guarantees *monotonicity preservation*: no spurious oscillations are introduced between waypoints. This is especially important near the descent phases where large changes in Z occur over a short segment; a standard cubic spline would generate overshoots below the floor level. Moreover, for segments where the pose is held constant, PCHIP reduces to a simple constant interpolation, while a natural cubic spline would still produce small oscillations around the constant value.

The interpolation is performed at a fixed time step $\Delta t = 1$ s, producing a sequence of $N = 100$ Cartesian reference samples.

6.2.3 CSV Export

The trajectory is exported to a comma-separated file with the following columns:

`Time, Cond, X, Y, Z, Roll, Pitch, Yaw`

where `Cond` is the boolean load flag (as a real $\{0,1\}$) associated with the time instant according to the segment it belongs to. The CSV file constitutes the sole interface between the offline Python planner and the 20-sim simulation environment.

6.3 DataFromFile Block and Reference Erogation

6.3.1 DataFromFile Block

The `EE_Trajectory` block (of type `DataFromFile`) reads the trajectory CSV at run time samplig at $\Delta t = 1$ s while the simulation integrator typically takes sub-second steps, the DLL performs zero-order hold interpolation between consecutive rows: the reference is held constant until the next data row is reached. This is consistent with the smooth PCHIP trajectory, which ensures that successive 1-second samples are already close to each other.

The `Cond` output is cast to a boolean via a `Cast` block (`output = (input == 1)`), which converts the real-valued $\{0, 1\}$ CSV column to the boolean type required by the `X0` and gravity blocks. A `Splitter` then fans the boolean out to the `X0` switching junction, the load gravity source `g`, and the `GravityCompensation` block.

6.3.2 Pose Assembly

The six scalar Cartesian channels are re-assembled into a six-element vector $[X,\, Y,\, Z,\, \phi,\, \theta,\, \psi]^T$ by a `Mux` block, whose output is fed to the `Planner` block.

The `Planner` block converts the Euler-angle representation of orientation into a full 4×4 homogeneous transformation matrix T_{des}, using the ZYX convention:

$$R_{\text{des}} = R_z(\psi)\, R_y(\theta)\, R_x(\phi), \tag{6.3}$$

$$T_{\text{des}} = \begin{bmatrix} R_{\text{des}} & \mathbf{p}_{\text{des}} \\ \mathbf{0}^T & 1 \end{bmatrix}, \tag{6.4}$$

where $\mathbf{p}_{\text{des}} = [X,\, Y,\, Z]^T$ is the desired position. T_{des} is then forwarded to the IK block.

6.4 Inverse Kinematics Block

6.4.1 UR5 Denavit–Hartenberg Parameters

The closed-form IK solution is based on the standard Denavit–Hartenberg parameterisation of the UR5. The DH parameters are summarised in table 6.2:

Table 6.2: UR5 Denavit–Hartenberg parameters.

Joint i	α_{i-1} [rad]	a_{i-1} [m]	d_i [m]	θ_i
1	0	0	$d_1 = 0.089159$	θ_1
2	$\pi/2$	0	0	θ_2
3	0	$a_2 = -0.425$	0	θ_3
4	0	$a_3 = -0.39225$	$d_4 = 0.10915$	θ_4
5	$\pi/2$	0	$d_5 = 0.09465$	θ_5
6	$-\pi/2$	0	$d_6 = 0.0823$	θ_6

6.4.2 Analytical Solution

The IK block implements the closed-form UR5 inverse kinematics. The desired end-effector transformation $T_{\text{des}} \in \text{SE}(3)$ is decomposed as:

$$T_{\text{des}} = \begin{bmatrix} n_x & o_x & a_x & p_x \\ n_y & o_y & a_y & p_y \\ n_z & o_z & a_z & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{6.5}$$

where $\mathbf{n}$, $\mathbf{o}$, $\mathbf{a}$ are the normal, orientation, and approach unit vectors of the tool frame, and $\mathbf{p}$ is the tool-centre-point position.

The six joint angles are solved in the following order:

Step 1 – Joint 1 (θ_1). The wrist-centre position $\mathbf{p}_{0,5}$ is obtained by offsetting the end-effector position backward along the approach axis by d_6:

$$p_{05x} = p_x - d_6\, a_x, \qquad p_{05y} = p_y - d_6\, a_y. \tag{6.6}$$

The shoulder angle is then:

$$R_s = \sqrt{p_{05x}^2 + p_{05y}^2}, \tag{6.7}$$

$$\alpha_1 = \text{atan2}(p_{05y},\ p_{05x}), \tag{6.8}$$

$$\alpha_2 = \arccos\!\left(\frac{d_4}{R_s}\right), \tag{6.9}$$

$$\theta_1 = \alpha_1 + c_s\, \alpha_2 + \frac{\pi}{2}, \tag{6.10}$$

where $c_s \in \{+1, -1\}$ is the shoulder configuration selector.

Step 2 – Joint 5 (θ_5). The wrist-flip angle follows directly from the projection of the end-effector position onto the $(\sin\theta_1, -\cos\theta_1)$ direction:

$$\theta_5 = c_w \arccos\!\left(\frac{p_x \sin\theta_1 - p_y \cos\theta_1 - d_4}{d_6}\right), \tag{6.11}$$

where $c_w \in \{+1, -1\}$ is the wrist configuration selector.

Step 3 – Joint 6 (θ_6). From the orientation columns:

$$\theta_6 = \text{atan2}\!\left(\frac{-o_x \sin\theta_1 + o_y \cos\theta_1}{\sin\theta_5},\ \frac{n_x \sin\theta_1 - n_y \cos\theta_1}{\sin\theta_5}\right). \tag{6.12}$$

Step 4 – Sum $\theta_{234} = \theta_2 + \theta_3 + \theta_4$. The combined angle of joints 2, 3, and 4 is obtained from the approach vector:

$$\theta_{234} = \text{atan2}(-a_z,\ -(a_x \cos\theta_1 + a_y \sin\theta_1)). \tag{6.13}$$

Step 5 – Joints 2 and 3 (θ_2, θ_3). An intermediate point $\mathbf{p}_{1,4}$ in the plane of the arm is computed:

$$p_{14x} = p_{05x}\cos\theta_1 + p_{05y}\sin\theta_1 - d_5\sin\theta_{234}, \tag{6.14}$$

$$p_{14z} = (p_z - d_6\,a_z) - d_1 + d_5\cos\theta_{234}. \tag{6.15}$$

The elbow angle θ_3 is found via the cosine rule:

$$D = \frac{p_{14x}^2 + p_{14z}^2 - a_2^2 - a_3^2}{2\,a_2\,a_3}, \qquad \theta_3 = \operatorname{atan2}\Big(c_e\sqrt{1-D^2},\ D\Big), \tag{6.16}$$

where $c_e \in \{+1, -1\}$ is the elbow configuration selector. The shoulder flexion θ_2 follows from:

$$\theta_2 = \operatorname{atan2}(p_{14z},\ p_{14x}) - \operatorname{atan2}(a_3\sin\theta_3,\ a_2 + a_3\cos\theta_3)\,. \tag{6.17}$$

Step 6 – Joint 4 (θ_4). The final angle is recovered from the sum constraint:

$$\theta_4 = \theta_{234} - \theta_2 - \theta_3. \tag{6.18}$$

The six joint angles are collected into the output vector:

$$\boldsymbol{\theta}_{\text{ref}} = [\theta_1,\ \theta_2,\ \theta_3,\ \theta_4,\ \theta_5,\ \theta_6]^T. \tag{6.19}$$

The three configuration selectors c_s, c_e, c_w are fixed parameters of the IK block, all set to $+1$ in the current model, selecting the right-shoulder, elbow-up, wrist-not-flipped solution.

6.4.3 Singularity Considerations

The solution for θ_6 involves a division by $\sin\theta_5$, which is undefined when $\theta_5 = 0$ (wrist singularity: the last two joints are aligned). In the current trajectory design, θ_5 remains sufficiently far from zero throughout the motion; no explicit singularity handling is required. Similarly, the elbow cosine D is kept within $[-1, 1]$ by design of the waypoints.

6.5 Phase Unwrapping

The analytical IK solution returns joint angles in $]-\pi, \pi]$ at every time step independently. If a joint crosses the $\pm\pi$ boundary between two consecutive time steps, the raw IK output exhibits a jump of 2π, which would cause a discontinuous (and physically unrealisable) reference for the PD controller.

The `Unwrap` block eliminates these jumps by enforcing continuity:

$$\theta_{\text{uw},i}(t) = \theta_{\text{raw},i}(t) - 2\pi\cdot\operatorname{round}\left(\frac{\theta_{\text{raw},i}(t) - \theta_{\text{prev},i}(t)}{2\pi}\right), \quad i = 1,\ldots,6, \tag{6.20}$$

where $\theta_{\text{prev},i}$ is the unwrapped joint angle at the *previous* time step, stored by a one-step `Delay` block. At initialisation, $\theta_{\text{prev},i}(0)$ is set to the IK solution of the initial pose, so that the first step is always consistent.

The round$(\cdot)$ function returns the nearest integer, ensuring that the correction added at each step is always the smallest integer multiple of 2π that brings the new raw angle closest to the previous unwrapped angle. This is the discrete-time equivalent of the standard continuous unwrapping condition $|\theta_{\text{uw}}(t) - \theta_{\text{prev}}(t)| < \pi$.

6.6 Feeding to Low-Level Control

The unwrapped joint reference vector $\boldsymbol{\theta}_{\text{ref}}(t)$ is fed to a `Demux` block that splits it into six scalar references $\theta_{\text{ref},i}(t)$, one per joint. Each scalar reference is routed to the `q_ref` input of the corresponding `PD` controller block.

The PD+G controller for joint i computes the motor torque command as:

$$\tau_i = K_{p,i}\big(\theta_{\text{ref},i} - q_i\big) + K_{d,i}\big(\dot{\theta}_{\text{ref},i} - \dot{q}_i\big) + \tau_{g,i}(\mathbf{q}), \tag{6.21}$$

where $\tau_{g,i}(\mathbf{q})$ is the gravity feedforward torque supplied by the `GravityCompensation` block. The desired joint velocity $\dot{\theta}_{\text{ref},i}$ is provided as a constant zero signal in the current implementation, meaning that the derivative action acts only on the measured joint velocity $\dot{q}_i$, effectively implementing a PD controller with a velocity feedback term.

The resulting torque command τ_i is scaled by the gear ratio N and injected into the harmonic drive sub-model as a modulated effort source (`MSe`), closing the control loop at the lowest level of the hierarchy.

The full signal chain from offline planning to motor torque is summarised in fig. 6.2:

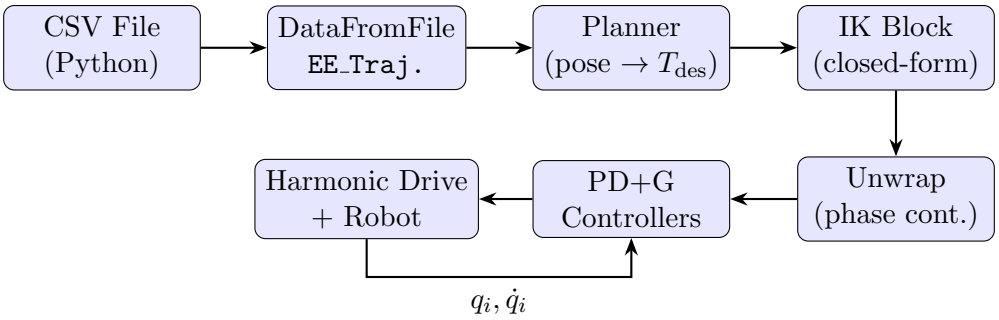


Figure 6.2: Signal chain from offline trajectory generation to motor torque command.

Chapter 7

Overall Robot Architecture

7.1 Hierarchical Structure

The complete `Robot` model assembles the sub-models described in the previous chapters into a hierarchical structure. Figure 7.1 provides a conceptual overview of the interconnections.

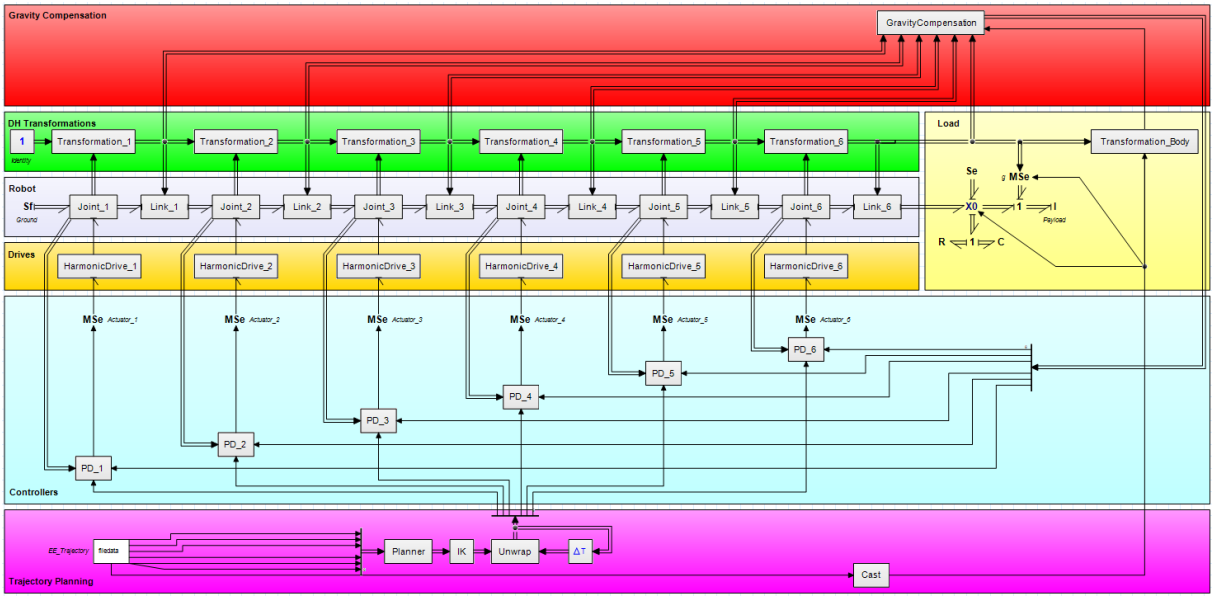


Figure 7.1: Full robot architecture.

7.2 Kinematic Chain Assembly

7.2.1 Robot Unit

The robot comprises identical units consisting of the Joint, Link, Transformation, Harmonic Drive, and a PD controller with gravity compensation, all interconnected sequentially.

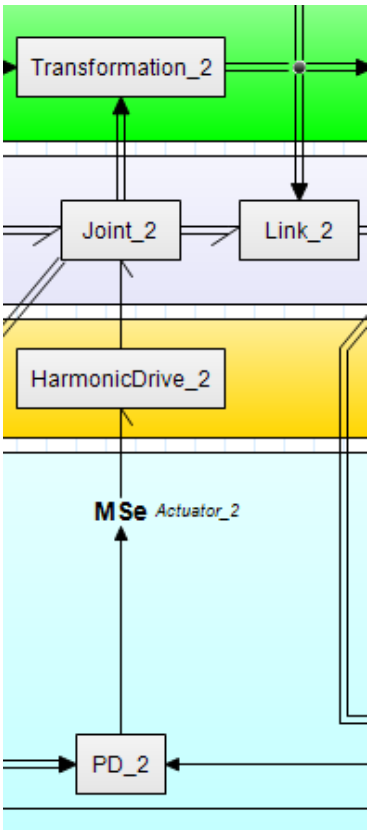


Figure 7.2: Robot unit architecture.

7.2.2 Power Bond Chain

The mechanical power flows from the base (world frame, fixed) through the joints and links in sequence. The base is connected to a *world bond*: a fixed-velocity source that sets the inertial frame velocity to zero ($S_f = 0$ for all six components).

For each link $i = 1, \ldots, 6$, the connectivity is defined as:

$$\text{Joint}_i \ (\texttt{twist_ipp}) \longrightarrow \text{Link}_i \ (\texttt{wrench_i}) \longrightarrow \text{Link}_i \ (\texttt{wrench_ipp}) \longrightarrow \text{Joint}_{i+1} \ (\texttt{twist_i}).$$

After Joint_6 and Link_6, the distal power port terminates at the load sub-system described in section 7.4.

7.2.3 Signal Chain for Transformations

The absolute transformation chain initializes with $T_0 = I_4$ (the identity matrix, representing the world frame).

$$T_0 \xrightarrow{\text{T_prev}} \text{Transf.}_1 \xrightarrow{\text{T_next}=T_1} \text{Transf.}_2 \xrightarrow{T_2} \cdots \xrightarrow{T_6} \text{GravityTerm}.$$

Each `Transformation` block also routes `T_next` to the `MSe` element of the corresponding `Link` block (for gravity vector orientation) and to the 3D visualization.

7.3 Global Causality Analysis

The Link sub-model acts as an **I-field** in integral causality; therefore, its outputs at both ports are flows. This matches the causality of the Joint sub-model, which instead outputs effort from both ports.

Consequently, the Joint block imposes the effort onto the robot chain, while the Links (representing the inertias) define the resulting velocities in response to this effort.

To manage these computational constraints:

- The world bond imposes velocity (effort-out causality at the base).
- At the base, a source of flow ($S_f = 0$) simulates a stationary ground.
- At the end-effector (a Link), the load is attached through a switching constraint block described in section 7.4.
- The harmonic drive outputs an effort, simulating a mechanical transmission.
- The motor driving the shaft is simulated as a source of effort.

Moreover, the non-linear C-element within the harmonic drive retains integral causality. No derivative causality is present in the final assembled model, ensuring the numerical stability of the constrained integration via BDF solvers.

7.4 Payload Model

7.4.1 Overview and Switchable Load

The payload sub-system has been redesigned to support two distinct operating modes: *free end-effector* (no load attached) and *rigid load attached*. The transition between the two modes is governed by a boolean condition signal `Cond`, which is read from the trajectory CSV file generated offline by the Python planner (see chapter 6). This allows the simulation to replicate a pick-and-place scenario in which the robot arm picks up a load at a specific instant, carries it along a portion of the trajectory, and then releases it.

The end of the kinematic chain (the distal port of Link_6) is connected to a `SwitchingZeroJunction` block (`X0`), which acts as the interface between the robot arm and the load sub-system.

The `X0` block has the following ports:

- `ee`: power input from Link_6 (the end-effector port of the last link).
- `one`: power output towards the payload I-element.
- `constrainer`: power port connected to the constraint 1-junction (feedback path).
- `source_effort`: power input from an S_e block (zero-effort source, modelling a free end-effector with no contact force applied).
- `condition`: boolean signal input from `Cond`.

7.4.2 Operating Modes

The switching logic of `X0` is governed by the following conditional equations:

$$e_{\text{ee}} = \begin{cases} e_{\text{constrainer}} & \text{if } \texttt{Cond} \\ e_{S_e} & \text{otherwise} \end{cases} \tag{7.1}$$

$$e_{\text{one}} = \begin{cases} e_{\text{constrainer}} & \text{if } \texttt{Cond} \\ \mathbf{0} & \text{otherwise} \end{cases} \tag{7.2}$$

$$f_{S_e} = \begin{cases} \mathbf{0} & \text{if } \texttt{Cond} \\ f_{\text{one}} - f_{\text{ee}} & \text{otherwise} \end{cases} \tag{7.3}$$

$$f_{\text{constrainer}} = \begin{cases} -(f_{\text{one}} - f_{\text{ee}}) & \text{if } \texttt{Cond} \\ \mathbf{0} & \text{otherwise} \end{cases} \tag{7.4}$$

Free mode (`Cond` = `false`). The end-effector sees the effort from the zero-effort source S_e (i.e. zero contact wrench). The payload I-element is decoupled: its effort input is set to zero and it does not exchange power with the chain. The constraint junction is also deactivated. This mode correctly simulates a free-hanging end-effector with no load attached.

Load mode (`Cond` = `true`). The end-effector effort is determined by the constraint, and the payload inertia is rigidly coupled to Link_6. The constraint junction (OneJunction_1) closes the loop: the flow mismatch between the payload and the end-effector is fed back and nullified by the constraint.

7.4.3 Rigid Constraint Between End-Effector and Payload

When the load is active, a rigid velocity constraint must be enforced between the distal port of Link_6 and the payload body. This is achieved via a 1-junction ($\texttt{OneJunction}_1$) connected to a very stiff C element (compliance $c = 10^{-6}$ m/N), which approximates a rigid kinematic constraint while preserving full integral causality:

$$\boldsymbol{\tau}_{\text{constraint}} = \frac{1}{c} \int \left(\mathbf{f}_{\text{one}} - \mathbf{f}_{\text{ee}} \right) dt. \tag{7.5}$$

The flow difference $\mathbf{f}_{\text{one}} - \mathbf{f}_{\text{ee}}$ integrates to a very small displacement (due to the high stiffness), and the resulting effort is fed back as the constraint wrench that enforces zero relative motion between the end-effector and the payload. An additional resistive element R ($r = 10^4$ Nm s/m) is placed on this same 1-junction to damp the high-frequency oscillations of the constraint spring and prevent numerical chattering.

This formulation avoids the introduction of algebraic constraints which is not ideal when switching between free and constrained modes, since it must be activated when the relative velocity is perfectly zero (at the pick-up instant) and deactivated when the relative velocity is also zero (at the release instant), but this is not guaranteed to be the case due to numerical precision issues.

7.4.4 Payload Inertia

The payload body is modelled as a full 6-DOF inertia element (I-field) with translational mass m_{load} and rotational inertia J_{load}:

$$\dot{\mathbf{p}} = \frac{1}{m_{\text{load}}}\big(\mathbf{f}_e^{[1:3]} - m_{\text{load}}\,\boldsymbol{\omega} \times \mathbf{v}\big), \tag{7.6}$$

$$\dot{\mathbf{h}} = J_{\text{load}}^{-1}\big(\mathbf{f}_e^{[4:6]} - \boldsymbol{\omega} \times J_{\text{load}}\boldsymbol{\omega}\big), \tag{7.7}$$

where $\mathbf{v}$ and $\boldsymbol{\omega}$ are the linear and angular velocities of the load frame, and $\mathbf{f}_e$ is the six-dimensional wrench exerted by the constraint. The default parameters are $m_{\text{load}} = 1.0$ kg and $J_{\text{load}} = \text{diag}(0.001, 0.001, 0.001)$ kg m^2.

7.4.5 Conditional Gravity on the Payload

A dedicated modulated source of effort `g` applies the gravitational wrench to the payload. It receives the same boolean `Cond` signal and the transformation matrix T_6 of the end-effector frame:

$$\mathbf{F}_{\text{grav}}^{[1:3]} = R_6^T \begin{bmatrix} 0 \\ 0 \\ -m_{\text{load}}\, g \end{bmatrix}, \qquad \mathbf{F}_{\text{grav}}^{[4:6]} = \mathbf{0}, \tag{7.8}$$

where R_6 is the rotation matrix of the end-effector frame extracted from T_6. When `Cond` is false, the gravitational effort is set to zero, consistent with the decoupled load.

The gravity compensation block `GravityCompensation` receives the same boolean `is_picked` flag and adds the load mass m_{load} to the effective mass of joint 6 when computing the feedforward gravity torques, ensuring that the PD+G controller correctly compensates for the additional weight whenever the load is picked up.

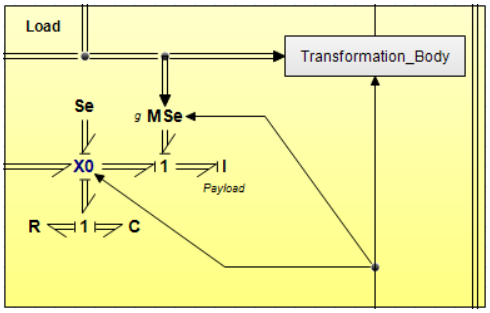


Figure 7.3: Switchable load architecture: the `X0` switching zero-junction controls the coupling between the end-effector and the payload I-element, driven by the trajectory `Cond` signal.

7.5 3-D Visualization

The 20-sim 3-D Mechanics toolbox receives the pose signals (`pose_abs`) and the rotation matrices from each `Transformation` block. STL bodies based on the UR5 model are utilized to represent each link. These bodies are positioned and oriented dynamically according to the offset positions and rotation matrices computed within the transformation blocks.

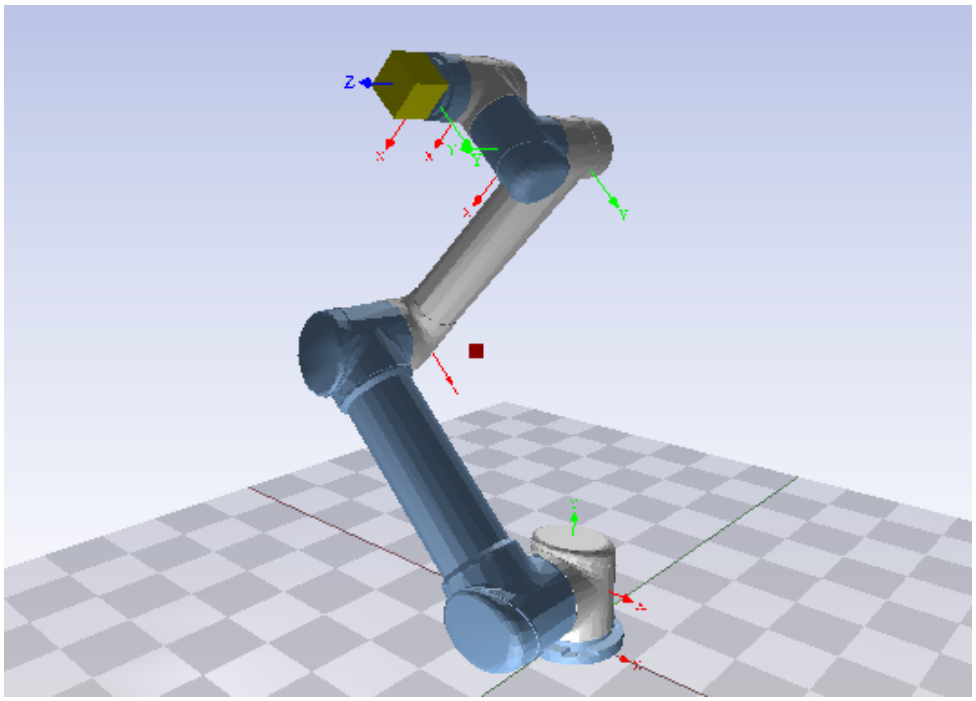


Figure 7.4: Robot 3D visualization.

Chapter 8

Simulation Results

8.1 Test Scenario

The simulation is designed to test all layers of the model simultaneously, covering both the dynamic response of the mechanical chain and the complete trajectory-tracking control stack:

1. **Cartesian trajectory tracking**: the end-effector is commanded to follow a smooth six-dimensional Cartesian reference trajectory (position and orientation) generated offline via a PCHIP spline interpolant through a set of ten waypoints spanning 100 seconds of simulation.

2. **Inverse kinematics**: at every time step, the desired end-effector pose is mapped to joint-space references by the closed-form UR5 IK solver. An unwrapping stage ensures continuity of the joint references across 2π boundaries.

3. **Gravity compensation**: the arm must hold its posture against gravity at all times, including while carrying a load.

4. **Harmonic drive effects**: the elastic flex-spline introduces a slight lag between the commanded and actual joint position.

5. **Switchable payload**: a rigid load of approximately 1 kg is attached to and detached from the end-effector at prescribed times, controlled by the `Cond` flag embedded in the trajectory CSV.

The joint initial conditions are set to match the IK solution at the first waypoint.

8.2 3-D Animation

Click the image Figure 8.1 to show the animation of the UR5 robot following its Cartesian trajectory.

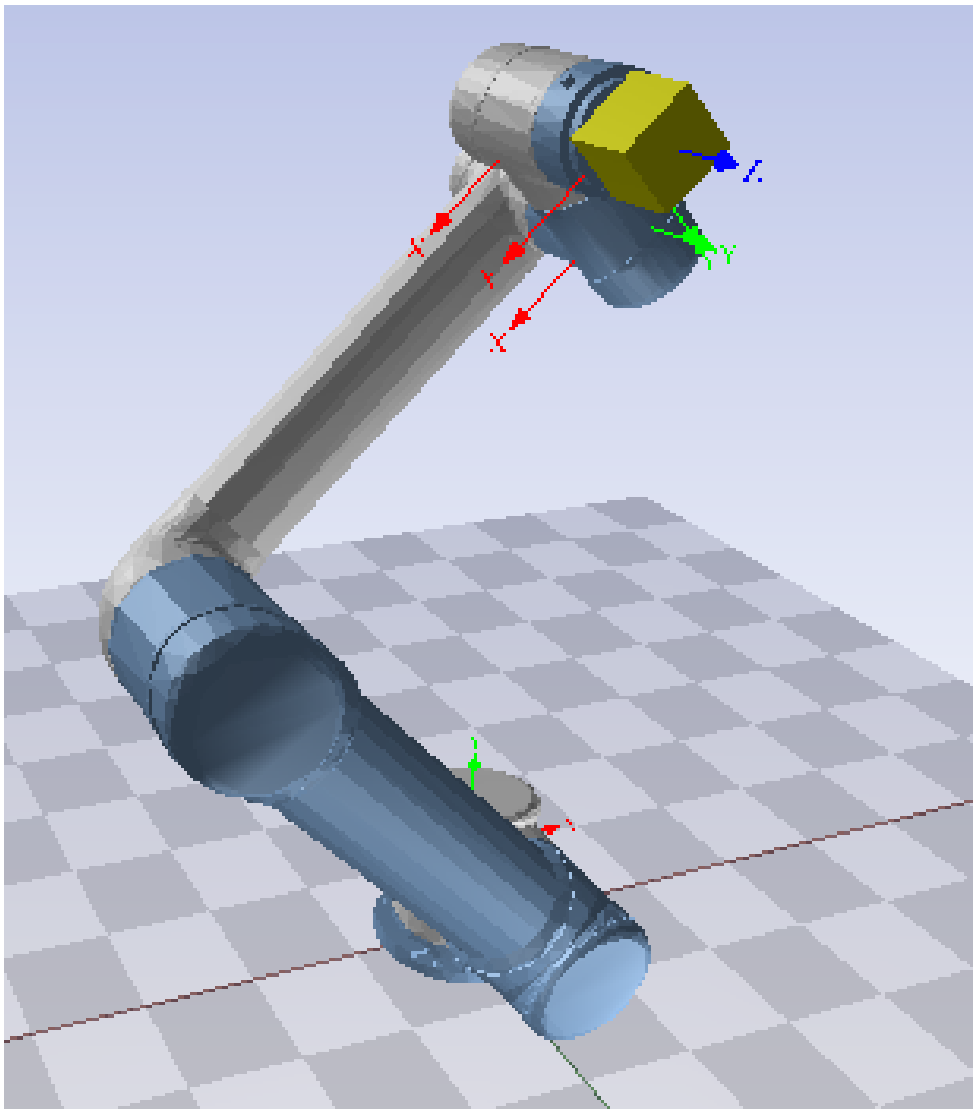


Figure 8.1: 3-D animation of the UR5 trajectory tracking.

8.3 Joint Position Tracking

Figure 8.2 shows the time evolution of all six joint angles $q_i(t)$ compared to their references $q_i^*(t)$.

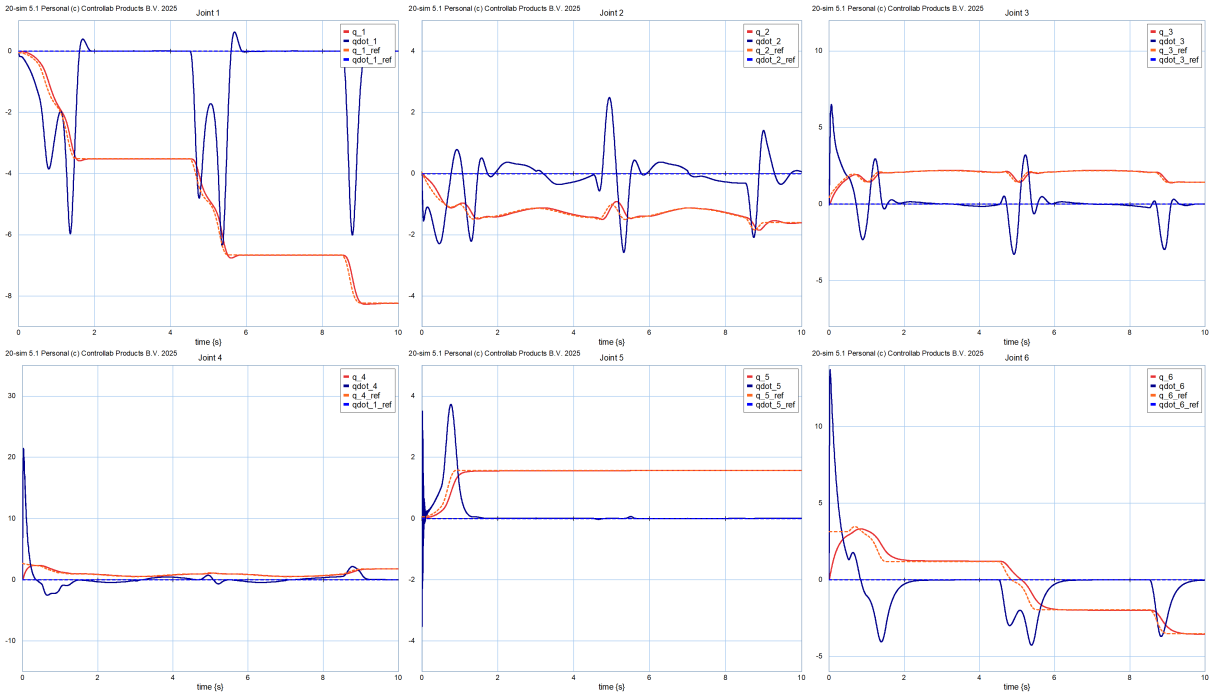


Figure 8.2: Joint position tracking

Observation. All six joints track their time-varying references with a small steady-state error, consistent with the PID control law. The tracking error remains bounded throughout the 10-second simulation, even across the load pick-up and release events. Due to the uncertainty in the load mass, and its compensation through the integral term only, a small error is noticeable during the pickup phase, which is then gradually reduced as the integral action accumulates, to later be fully compensated at steady state. No overshoots are present thanks to the derivative damping term and the gravity feedforward.

The delay between the reference and actual joint angles is around 0.1 seconds.

8.4 Joint Torques

Figure 8.3 shows the commanded torques $\tau_i(t)$ for all joints.

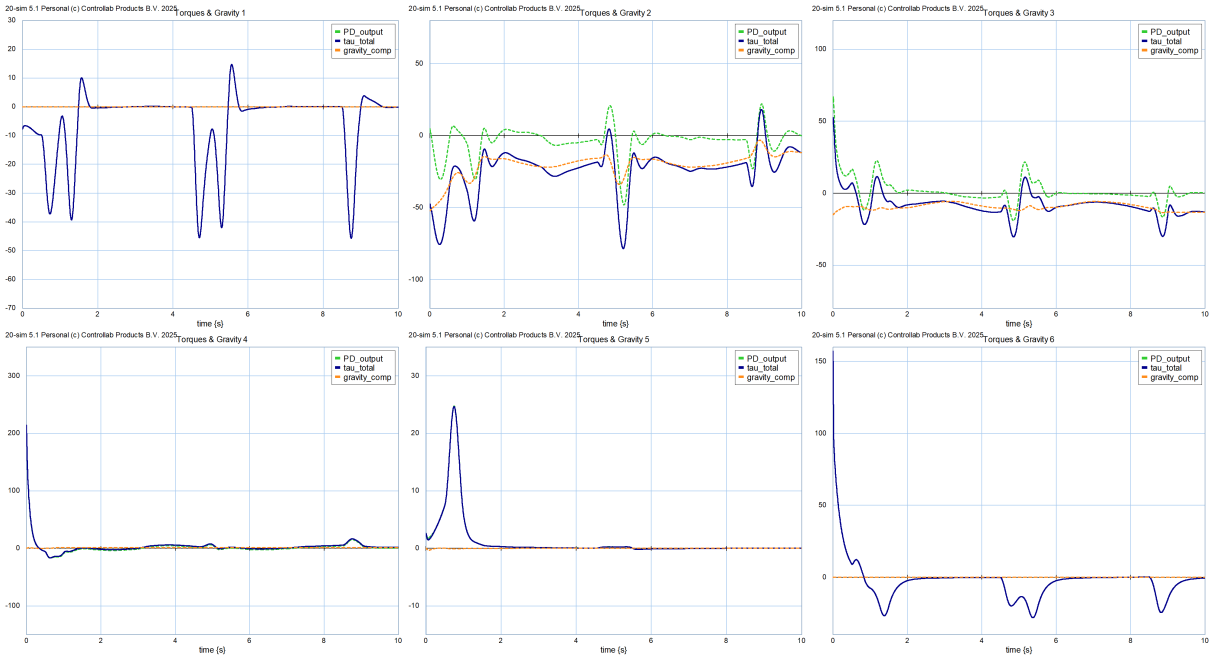


Figure 8.3: Joint control torques $\tau_i(t)$

Observation. Torques are smooth and bounded with no sign of instability. As the trajectory time is 10s, the dynamics is not negligible and therefore the PID is very active compared to a pure position control scenario, where the torques would be mostly constant and equal to the gravity compensation term.

The close match between the steady-state commanded torque and the analytically predicted gravity torque confirms that the compensation term is correctly implemented.

8.5 Harmonic Drive Elastic Deflection

Figure 8.4 shows the elastic deflection $\varphi_i = q_{m,i}/N - q_i$ in the flex-spline of each harmonic drive.

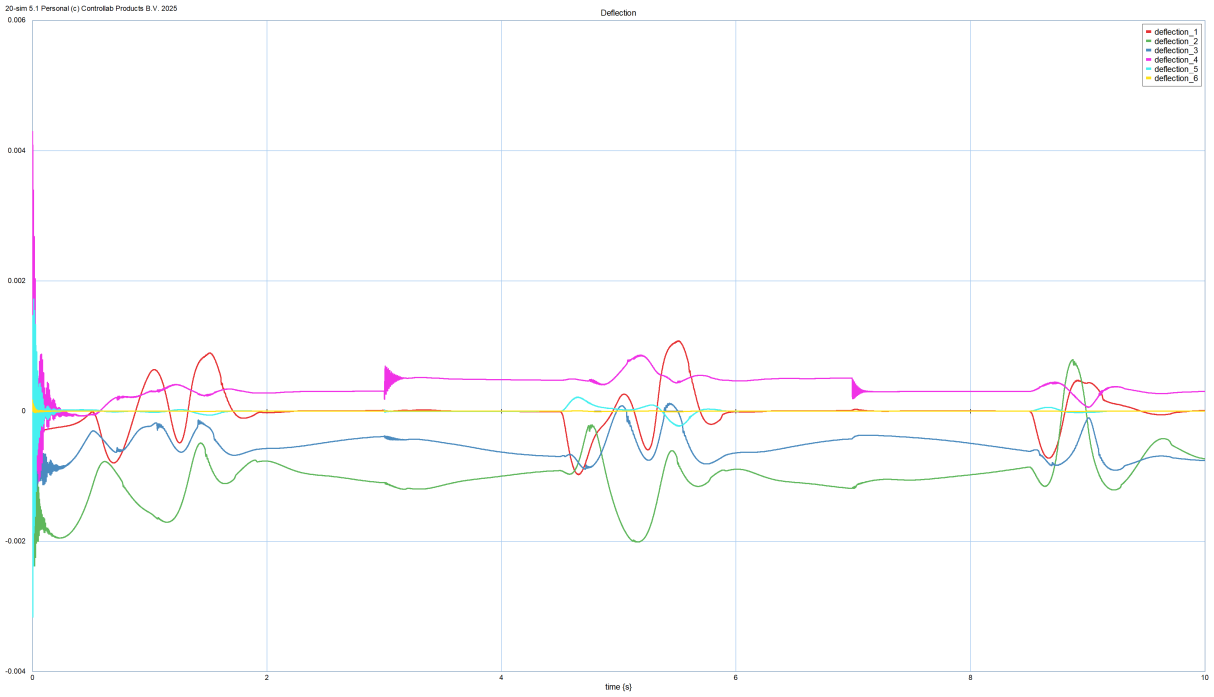


Figure 8.4: Elastic twist deflection $\varphi_i(t)$ in the harmonic drive flex-spline.

Observation. Peak deflections occur during the transient and are bounded. The added structural damping R element on the compliance branch effectively prevents sustained oscillations: the deflection waveforms are smooth and decay rapidly after each change in the reference trajectory.

8.6 End-Effector Trajectory: Reference vs. Actual

Figure 8.5 shows the end effector position in the world reference frame.

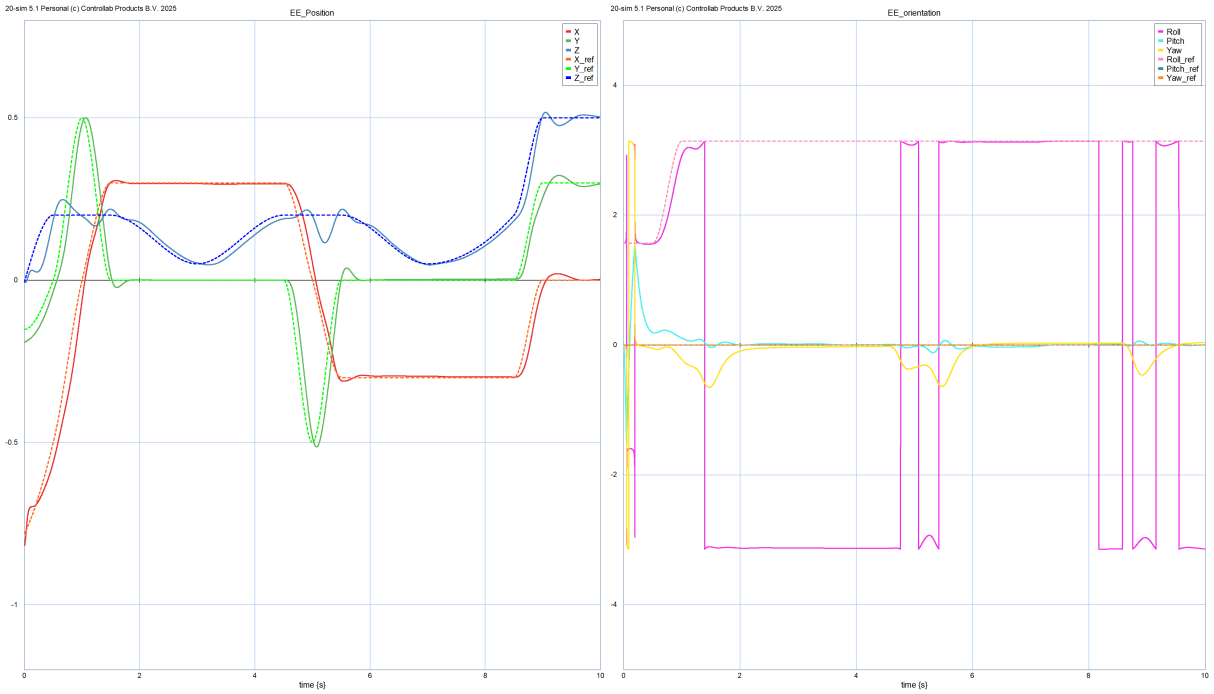


Figure 8.5: End effector position in the world reference frame

Observation. The actual end-effector position closely tracks the Cartesian reference generated by the PCHIP spline, with a small but non-zero lag attributable to the finite PD gains. The delay between the reference and actual end-effector position is around 0.4 seconds, while the error magnitude is $< 0.5\%$

8.7 End-Effector Position: Kinematics vs. Dynamics

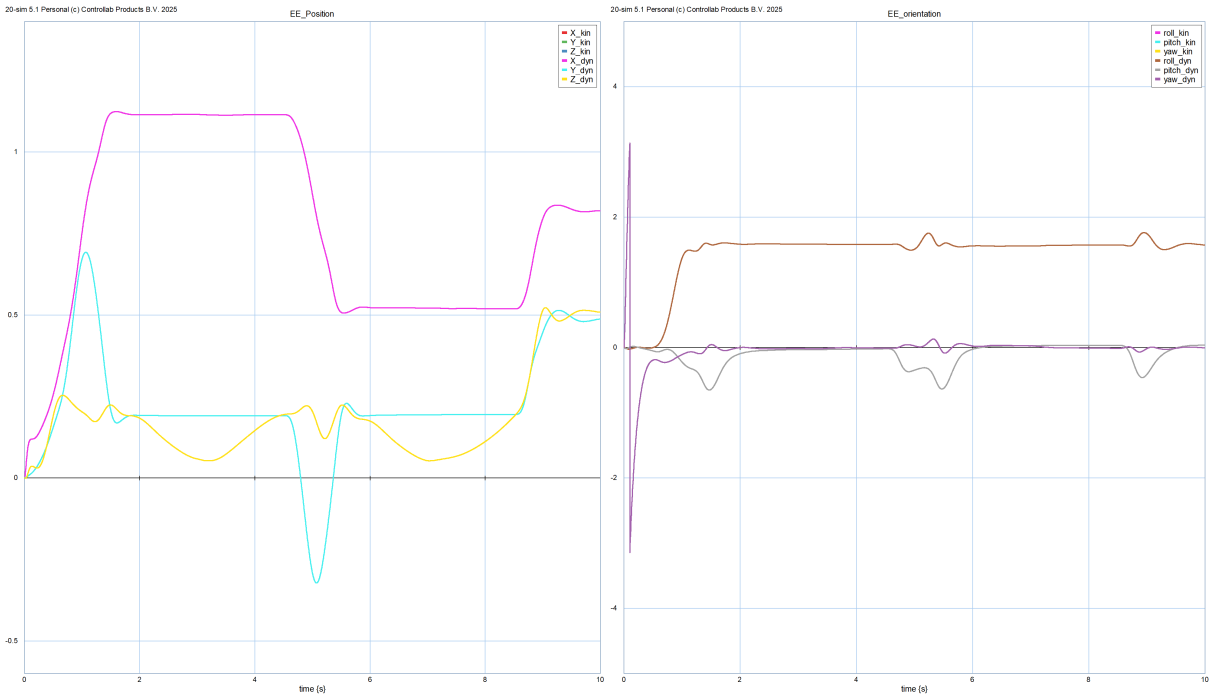


Figure 8.6: comparison of end-effector XYZ position computed (i) directly from the bond-graph state via the DH forward kinematics chain (Transformation blocks) and (ii) from the body velocity integration in the Link sub-models

Observation. The end-effector position computed by forward kinematics through the Denavit–Hartenberg transformation chain and the position obtained by integrating the bond-graph body velocities within the Link sub-models are in very close agreement throughout the simulation, with a discrepancy of the order of numerical precision (below 10^{-8} m). This internal consistency check confirms that the MTF-based wrench/twist propagation and the DH transformation chain are mutually coherent: the kinematic and dynamic layers of the model are correctly coupled.

8.8 Load–End-Effector Constraint Verification

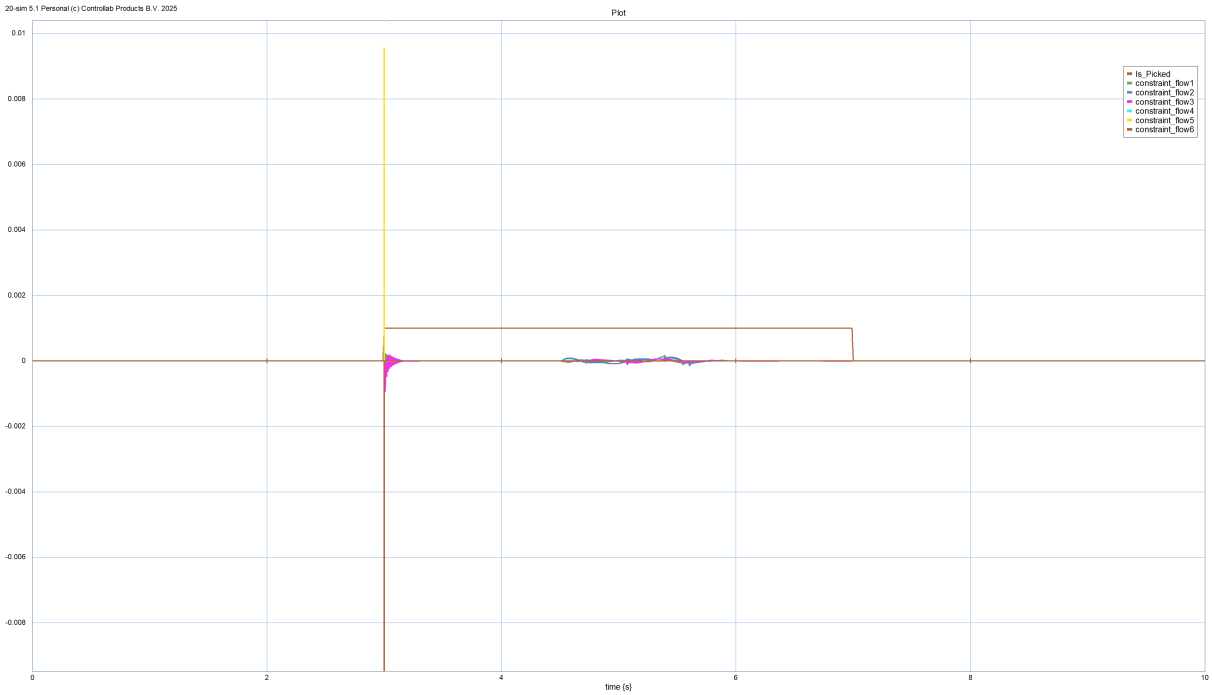


Figure 8.7: time evolution of the six-component constraint wrench at the X0 junction, and of the velocity difference between the end-effector port and the payload body during the load-attached phase

Observation. During the load-attached phase (controlled by the `Cond` flag), the velocity difference between the end-effector port and the payload body remains at the level of numerical noise ($< 10^{-3}$ m/s), confirming that the very stiff C element ($c = 10^{-6}$) effectively enforces a rigid connection. The constraint wrench correctly reflects the inertial and gravitational load of the 1 kg payload: during acceleration phases the force component along the direction of motion increases accordingly, while at near-constant velocity the wrench is dominated by the gravity contribution. At the load pick-up and release instants a brief transient is visible in the constraint wrench as the stiff spring absorbs the initial velocity mismatch; the additional R damper at the constraint junction suppresses this transient within a few simulation steps.

8.9 Conclusions

A complete bond-graph model of the UR5 six-degree-of-freedom robot manipulator has been presented and validated. Starting from the fundamental Newton–Euler equations of a rigid body expressed in terms of twists and wrenches, a hierarchical set of reusable 20-sim sub-models — `Link`, `Joint`, `Transformation`, `HarmonicDrive`, `PD`, and `GravityTerm` — was developed and assembled into a complete closed-loop trajectory tracking system.

The following aspects deserve particular emphasis:

- The **Modulated transformer** (MTF) is the central bond-graph element for both the change-of-reference-point in the Link model and the DH transformation in the Joint model.

- **Causality assignment** is the most challenging design step and it's possible to maintain a full integral causality of all the elements thanks to constraint blocks, used to interface two rigid bodies.
- The **GravityTerm** block, while signal-only, is critical for controller stability: without feedforward compensation, the PD controller cannot hold arbitrary postures.
- The **structural damping** added in parallel with the flex-spline compliance is necessary to obtain physically plausible and numerically well-behaved harmonic drive dynamics; without it, the elastic mode persists indefinitely.
- The **switchable load** mechanism based on the `SwitchingZeroJunction` and the trajectory `Cond` flag elegantly handles the pick-and-place scenario while preserving integral causality throughout, requiring no modification of the underlying bond-graph topology.
- The **closed-form IK solver** combined with a phase-unwrapping stage provides smooth, continuous joint references directly from the Cartesian trajectory, closing the loop between the high-level task-space planner and the joint-level PD+G controller.